

dataset/kuaiREC/kuaiREC_process.py (Q&A Documentation)

Q: What does this file do? **A:** This is a preprocessing script that merges raw KuaiRec CSVs, engineers/normalizes labels and durations, builds vocabularies for sparse and sequence features, constructs a feature schema, creates train/test splits, and writes the final pickle used by training.

Q: Why is the output schema (description) important? **A:** All models and the dataloader depend on the description list to decide feature types and vocab sizes. If it is wrong, embeddings and tensors will not line up.

Q: Why does this script also generate D2Q artifacts? **A:** D2Q needs a duration bucket index and per-bucket play-time quantiles to map raw labels to a quantile target and back. The script produces these CSVs for reproducible D2Q training.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
# (no top-level classes/functions detected)
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import pandas as pd
import numpy as np
import pickle
```

```
with open("row_data/kuaiREC_caption_category.csv", "r", encoding="utf-8", errors="ignore") as f:
    lines = f.readlines()
```

```
with open("row_data/cleaned_kuaiREC_caption_category.csv", "w", encoding="utf-8") as f:
    f.writelines(lines)
```

```
df5 = pd.read_csv("row_data/cleaned_kuaiREC_caption_category.csv", encoding="utf-8")
df5["video_id"].astype(int)
df5.to_csv("row_data/cleaned_kuaiREC_caption_category.csv", index=False, encoding="utf-8")
```

```
#join feature files to one file
df1 = pd.read_csv("row_data/big_matrix.csv")
df2 = pd.read_csv("row_data/user_features.csv")
df3 = pd.read_csv("row_data/item_daily_features.csv").loc[:, ['video_id', 'video_type', 'music_id', 'video_id']]
df3 = df3.drop_duplicates(subset=['video_id'])
```

```

df4 = pd.read_csv("row_data/item_categories.csv")
df5 = pd.read_csv("row_data/cleaned_kuairec_caption_category.csv", encoding="utf-8").loc[:, ['video_id',

df_merged_1_2 = pd.merge(df1, df2, on="user_id", how="left")
df_merged_1_2_3 = pd.merge(df_merged_1_2, df3, on="video_id", how="left")
df_merged_1_2_3_4 = pd.merge(df_merged_1_2_3, df4, on="video_id", how="left")
df_final = pd.merge(df_merged_1_2_3_4, df5, on="video_id", how="left")

df =df_final.fillna('12345')

#filter AD video
df = df[df['video_type'] != 'AD']
df = df[df['play_duration'] > 0]
df = df[df['video_duration'] > 0]

df = df.sample(frac=0.1, random_state=312).reset_index(drop=True)

#delete unnecessary features
df = df.drop(columns=['video_type','time','date','watch_ratio','follow_user_num','fans_user_num','friend

# indentify different types of feature
label="play_duration"
sequence_feature="feat"
sparse_feature_list=['feat','user_id','video_id','music_id','video_tag_id','user_active_degree','is_low
dense_feature_list=['video_duration','timestamp']

# create feature desc and process play_time_ms/duration_ms
desc=[('play_time', -1, 'label'), ('duration', -1, 'ctn')]
df['play_time'] = df['play_duration'] / df['play_duration'].max()

```

Q: Example: end-to-end data flow involving this file **A: Core pipeline shape (schematic):**

See the run scripts for end-to-end usage.

Q: Full source code of the Python file **A: The complete file is included verbatim for reference.**

```

import pandas as pd
import numpy as np
import pickle

with open("row_data/kuairec_caption_category.csv", "r", encoding="utf-8", errors="ignore") as f:
    lines = f.readlines()

with open("row_data/cleaned_kuairec_caption_category.csv", "w", encoding="utf-8") as f:
    f.writelines(lines)

df5 = pd.read_csv("row_data/cleaned_kuairec_caption_category.csv", encoding="utf-8")
df5["video_id"].astype(int)
df5.to_csv("row_data/cleaned_kuairec_caption_category.csv", index=False, encoding="utf-8")

#join feature files to one file
df1 = pd.read_csv("row_data/big_matrix.csv")
df2 = pd.read_csv("row_data/user_features.csv")
df3 = pd.read_csv("row_data/item_daily_features.csv").loc[:, ['video_id', 'video_type','music_id','video_tag_id']]
df3 = df3.drop_duplicates(subset=['video_id'])

```

```

df4 = pd.read_csv("row_data/item_categories.csv")
df5 = pd.read_csv("row_data/cleaned_kuairec_caption_category.csv", encoding="utf-8").loc[:,['video_id', 'first_level_category_id']]

df_merged_1_2 = pd.merge(df1, df2, on="user_id", how="left")
df_merged_1_2_3 = pd.merge(df_merged_1_2, df3, on="video_id", how="left")
df_merged_1_2_3_4 = pd.merge(df_merged_1_2_3, df4, on="video_id", how="left")
df_final = pd.merge(df_merged_1_2_3_4, df5, on="video_id", how="left")

df = df_final.fillna('12345')

#filter AD video
df = df[df['video_type'] != 'AD']
df = df[df['play_duration'] > 0]
df = df[df['video_duration'] > 0]

df = df.sample(frac=0.1, random_state=312).reset_index(drop=True)

#delete unnecessary features
df = df.drop(columns=['video_type', 'time', 'date', 'watch_ratio', 'follow_user_num', 'fans_user_num', 'friend_user_num', 'register_day'])

# identify different types of feature
label="play_duration"
sequence_feature="feat"
sparse_feature_list=['feat', 'user_id', 'video_id', 'music_id', 'video_tag_id', 'user_active_degree', 'is_lowactive_period', 'is_live_stream']
dense_feature_list=['video_duration', 'timestamp']

# create feature desc and process play_time_ms/duration_ms
desc=[('play_time', -1, 'label'), ('duration', -1, 'ctn')]
df['play_time'] = df['play_duration'] / df['play_duration'].max()
df['duration'] = df['video_duration'].clip(upper=df['play_duration'].max()) / df['play_duration'].max()
df = df[df['play_time'] < df['duration']*10]
df['play_time'] = df.apply(lambda row: row['play_time'] if row['play_time'] < row['duration']*10 else row['duration']*10, axis=1)
df['play_time'] = df['play_time'].astype(float)
df['duration'] = df['duration'].astype(float)

# preprocess sparse features:
for sparse_feature_name in sparse_feature_list:
    sparse_feature_set= set()
    sparse_feature_col = []
    for index, row in df.iterrows():
        if (sparse_feature_name == sequence_feature):
            sparse_feature = [str(i) for i in row[sparse_feature_name].split(",")]
            sparse_feature_set.update(sparse_feature)
            sparse_feature_col.append(sparse_feature)
        else:
            sparse_feature = str(row[sparse_feature_name])
            sparse_feature_set.add(sparse_feature)
            sparse_feature_col.append(sparse_feature)

# generate the vocabulary of search tokens and items
sparse_feature_voc = {word: idx for idx, word in enumerate(sparse_feature_set)}
print("sparse feature {} size: {}".format(sparse_feature_name, len(sparse_feature_voc)))

# reset query and item index with new vocabulary
for idx, ori_feature in enumerate(sparse_feature_col):
    if (sparse_feature_name == sequence_feature):
        sparse_feature_col[idx] = [sparse_feature_voc[word] for word in ori_feature]
    else:
        sparse_feature_col[idx] = sparse_feature_voc[ori_feature]

if(sparse_feature_name == sequence_feature):
    sparse_mask_col= []
    for idx, ori_feature in enumerate(sparse_feature_col):
        feature_len = len(ori_feature[:10])
        new_feature = ori_feature[:10] + [0]*(10-feature_len)
        feature_mask = [1.0]*feature_len + [0.0]*(10-feature_len)

```

```

        sparse_feature_col[idx] = new_feature
        sparse_mask_col.append(feature_mask)
    df['featmask'] = sparse_mask_col
    df[sparse_feature_name] = sparse_feature_col

    # delete features that only has one value and adding desc
    if len(sparse_feature_voc) == 1:
        df = df.drop(sparse_feature_name, axis=1)
        print("remove the {} feature !!!!!".format(sparse_feature_name))
    else:
        if (sparse_feature_name == sequence_feature):
            desc.append((sparse_feature_name, len(sparse_feature_voc), 'seq'))
            desc.append(('featmask', -1, 'seqm'))
        else:
            desc.append((sparse_feature_name, len(sparse_feature_voc), 'spr'))

# generate precomputed duration buckets for D2Q
n_bins = 50
df['duration_bucket'], bins = pd.qcut(df['duration'],q=n_bins,labels=False,retbins=True,duplicates='drop' )
bucket_ranges = pd.DataFrame({'bucket_index': range(len(bins)-1),'min_duration': bins[:-1],'max_duration': bins[1:]})
bucket_ranges.to_csv('d2q_duration_bucket_ranges.csv', index=False)
desc.append(('duration_bucket', len(bins) - 1, 'spr'))

# split train, test set
df = df.sample(frac=1, random_state=1234).reset_index(drop=True)
df_train = df[:int(0.8*len(df))]
df_test = df[int(0.8*len(df)):]

# generate precomputed quantiles in each duration bucket for D2Q
quantile_num= 100
quantiles = np.linspace(0, 1, quantile_num+1)
quantile_df = (df_train.groupby('duration_bucket')['play_time'].quantile(quantiles).reset_index().rename(columns={'level_1': 'q'}))
quantile_pivot = quantile_df.pivot(index='duration_bucket',columns='quantile',values='play_time')
quantile_pivot.to_csv('d2q_duration_bucket_playtime_quantiles.csv')

# save to pickle
data = {
    "train": df_train,
    "test": df_test,
    "description": desc
}
with open('./kuaiREC_data.pkl', 'wb+') as f:
    pickle.dump(data, f)

```